Theoretical Knowledge: Advanced Log Analysis

1. Core Concepts

1.1 Log Correlation

Log correlation is the process of linking and analyzing log entries from multiple, disparate sources — such as firewalls, endpoint detection systems, application servers, and authentication systems — to identify attack patterns that would be invisible when examining any single log source in isolation.

How It Works:

Log correlation engines (built into SIEMs like Elastic Security, Splunk, or QRadar) ingest normalized logs and apply correlation rules. These rules define conditions across multiple log types that, when satisfied together within a time window, trigger a correlated alert. **Example: Linking Failed Logins with Suspicious Outbound Traffic**

Consider the following scenario:

1. **Windows Security Log** records multiple Event ID 4625 (Failed Logon Attempt) entries from workstation `WS-PC01` within a 5-minute window.
2. **Firewall Log** records unusual outbound connections from `WS-PC01` to an external IP `185.220.101.45` on port 443, shortly after the failed logon burst.
3. **DNS Log** shows `WS-PC01` resolving a domain `c2-relay.darknet[.]xyz` — a known command-and-control (C2) domain.

Individually, failed logins might be a user mistyping their password. Outbound HTTPS traffic is normal. But **correlated together**, this pattern strongly suggests:

- An attacker is brute-forcing credentials on `WS-PC01` .
- The machine may already be compromised (C2 communication is active).
-

The failed logins could be lateral movement attempts after initial compromise.

Correlation Rule (Pseudo-logic):

```
IF (Event ID 4625 count > 5 within 5 minutes from Source X)
AND (Outbound connection from Source X to external IP on non-standard port OR known ba
WITHIN 10-minute window
THEN → Generate HIGH severity alert: "Potential Brute Force + C2 Activity"
```

Key Log Sources for Correlation:

Log Source	Key Events	Use Case
Windows Security Logs	4625 (Failed Login), 4624 (Successful Login), 4672 (Special Privileges)	Authentication monitoring
Firewall Logs	Allow/Deny rules, NAT translations	Network perimeter monitoring
DNS Logs	Query records, NXDOMAIN responses	C2 detection, data exfiltration
Proxy/Web Logs	URL requests, user-agent strings	Malware downloads, beaconing
Endpoint (EDR) Logs	Process creation, file modifications	Malware execution, lateral movement
Application Logs	Error codes, access patterns	Application-layer attacks

1.2 Anomaly Detection

Anomaly detection involves identifying deviations from established baselines of "normal" activity. Unlike signature-based detection (which matches known patterns), anomaly detection can catch novel threats — zero-day exploits, insider threats, or advanced persistent threats (APTs) — that don't match any known signature.

Techniques:

Technique	Description	Example
Statistical Methods	Uses mean, standard deviation, and thresholds to flag outliers	A user who typically transfers 50MB/day suddenly transfers 5GB
Rule-Based Methods	Predefined rules based on domain knowledge	Login from a country where the organization has no employees
Machine Learning	Models learn normal behavior and flag deviations	Elastic ML detects a user accessing 500 files in 10 minutes (baseline: 20 files)
Time-Series Analysis	Tracks metrics over time to detect temporal anomalies	DNS query volume spikes at 3 AM on a workstation

Common Anomaly Indicators:

- **Unusual login times**: A user who always logs in 9 AM–5 PM suddenly authenticates at 2 AM.
- **High-volume data transfers**: > 1 GB outbound transfer from a workstation in under 1 minute.
- **Geographic impossibility**: Login from India at 10:00 AM, followed by a login from Russia at 10:15 AM (impossible travel).
- **New process execution**: A never-before-seen process (`mimikatz.exe`) runs on a server.
- **Lateral movement patterns**: One account authenticating to 15 different workstations in rapid succession.

Elastic Security Anomaly Detection Example:

Elastic Machine Learning jobs can be configured to detect anomalies automatically:

```
{
  "job_id": "high_bytes_out_detection",
  "analysis_config": {
    "bucket_span": "1m",
    "detectors": [
      {
        "function": "high_sum",
        "field_name": "destination.bytes",
        "over_field_name": "source.ip"
      }
    ]
  },
  "data_description": {
    "time_field": "@timestamp"
  }
}
```

This job monitors total outbound bytes per source IP per minute and flags anomalies when any IP sends significantly more data than its historical average.

1.3 Log Enrichment

Log enrichment is the process of augmenting raw log data with additional contextual information to improve the speed and accuracy of analysis. Raw logs contain IP addresses, timestamps, and event codes — but lack the context needed for rapid decision-making.

Types of Enrichment:

Enrichment Type	What It Adds	Example
GeoIP	Country, city, latitude/longitude for IP addresses	IP `185.220.101.45` → Tor Exit Node, Germany

Threat Intelligence	Reputation scores, IOC matches	IP matches AlienVault OTX known C2 feed
Asset Information	Hostname, owner, business unit, criticality	IP `10.0.1.55` → `HR-Server-01` , Critical Asset
User Context	Role, department, access level	User `jsmith` → Finance Department, Privileged Account
DNS Reverse Lookup	Domain name for IP addresses	`93.184.216.34` → `example.com`
WHOIS Data	Domain registration, owner, creation date	Domain registered 3 days ago → Suspicious

Why Enrichment Matters:

Without enrichment, an analyst sees:

```
2025-08-18 12:05:00 | 4625 | src:192.168.1.100 | dst:185.220.101.45
```

With enrichment, the same log becomes:

```
2025-08-18 12:05:00 | 4625 (Failed Login) | src:192.168.1.100 (HR-Workstation-05, jsmi
| dst:185.220.101.45 (Tor Exit Node, Germany, Malicious - OTX Score: 9/10)
```

The enriched log immediately tells the analyst: a Finance user's workstation is communicating with a known Tor exit node after a failed login — this is high-priority.

GeoIP Enrichment in Elastic (Logstash Pipeline):

```
filter {
  geoip {
    source => "destination.ip"
    target => "destination.geo"
    database => "/usr/share/GeoIP/GeoLite2-City.mmdb"
  }
}
```

2. Key Objectives

- Develop skills to analyze and correlate logs from diverse sources to uncover complex, multistage threats.
- Reduce false positives by applying contextual enrichment and correlation — ensuring analysts focus on real threats.
- Build proficiency in configuring anomaly detection rules (both statistical and ML-based) in SIEM platforms.
- Master the art of translating raw log data into actionable security intelligence.

3. Study Resources

Resource	Focus Area	Link
SANS Reading Room — "Effective Log Analysis"	Log correlation techniques, best practices	https://www.sans.org/white-papers/
Elastic Documentation — Anomaly Detection	ML-based anomaly detection setup and configuration	https://www.elastic.co/guide/en/machinelearning/

CISA — Equifax Breach Case Study	Real-world log correlation failure analysis	https://www.cisa.gov/news-events
NIST SP 800-92 — Guide to Log Management	Log management standards and frameworks	https://csrc.nist.gov/publications/detail/sp/800-92/final
Sigma Rules Repository	Open-source detection rules for SIEMs	https://github.com/SigmaHQ/sigma

4. Real-World Case Study: Equifax Breach (2017)

The Equifax breach exposed 147 million records. A key failure was **inadequate log monitoring**:

- An expired SSL certificate on a network inspection device meant **outbound traffic was not being monitored** for 19 months.
- Apache Struts vulnerability (CVE-2017-5638) was exploited, but **no log correlation rule** linked the web application error logs with the outbound data exfiltration.
- If log correlation had been properly configured (linking application errors + anomalous outbound traffic + new processes), the breach could have been detected within hours, not months.

Lesson: Log correlation is not optional — it is the foundation of effective threat detection.

Document prepared as part of SOC Analyst Training — Theoretical Knowledge Module